MODULE 06 · DOMAIN 1 - PLAN AND MANAGE AN AZURE AI SOLUTION

Quotas, rate limits, scaling and cost

How quota is assigned, why 429s happen, spillover, prompt caching, gateway limits and the cost footprint of agents.

DOMAIN 1 - Plan and manage WEIGHT 25-30% MODULE 06 of 39

admin@sthithapragna.com

Quota is what you are allowed. **Rate limits** are how fast you may spend it. **Cost** is what you actually spent. Three different numbers.

How quota is assigned

- Quota is set per **subscription, region, model and deployment type**, in tokens per minute.
- You **assign TPM to deployments** until the pool is used: one 240K deployment, or two of 120K.
- A deployment's TPM sets its **token limit and its requests-per-minute limit**. The ratio varies by model.
- TPM quota has **nothing to do with** a model's maximum input size.
- Quota increases favour customers already using their existing quota.

> ON THE EXAM
>
> 'Approved quota at subscription level, but still throttled' often means the TPM was never assigned to that deployment.

How a request is counted, and when

COUNTER	WHAT IT COUNTS	WHEN
Token rate limit	Prompt plus max_tokens, an estimate	As the request arrives
Request rate limit	Number of requests, in 1 to 10 second windows	As the request arrives
Billing and usage metrics	Actual tokens processed, successful requests	After processing finishes

ON THE EXAM

Failed requests still count against rate limits. Hammering retries makes throttling worse.

Throttled while usage looks low? Check **max_tokens** and **bursts** before asking for more quota.

Not every 429 means the same thing

KIND OF 429	TELL-TALE SIGN	WHAT TO DO
Rate limit exceeded	'Rate limit is exceeded'	Assign or rebalance TPM; request more quota
Capacity throttling	'High demand' message	Retry after the delay; consider provisioned
Temporary adjustment	Header limit below your configured TPM	Back off; it resolves within hours
Parameter inflation	Throttled, yet usage metrics are low	Lower max_tokens to realistic size

ON THE EXAM

Read the error message and the x-ratelimit headers before choosing a fix. Each kind has a different remedy.

Retrying correctly

- Honour the **retry-after-ms** header when it is present.
- Otherwise use **exponential backoff with random jitter**, and cap the number of retries.
- The OpenAI Python SDK already retries 429s, **twice by default**, with backoff.
- Adding your own retry loop on top multiplies requests. Set the SDK's retries to zero if you do.
- Watch the **x-ratelimit-remaining** headers to slow down before you are throttled.

> ON THE EXAM
>
> Retrying immediately, in a tight loop, is always a wrong answer. Unsuccessful requests still count.

Standard and provisioned throttle differently

STANDARD DEPLOYMENTS

Counters

- TPM and RPM counters per deployment
- Shared pool across customers
- Limits can be temporarily reduced
- Usage tiers affect latency at high volume

PROVISIONED DEPLOYMENTS

Utilisation

- Throttles only at 100% utilisation
- Returns 429 at once, with retry-after-ms
- Estimate: prompt minus cached tokens, plus max_tokens
- Cached tokens do not count toward utilisation

ON THE EXAM

On provisioned, a realistic max_tokens directly raises how many calls can run at once.

Spillover: the overflow hotel

- When a provisioned deployment is full, **spillover** sends the overflow to a **standard deployment**.
- The standard deployment must be in the **same Foundry resource**.
- Triggers on a 429 from full PTUs, and on a 400 for context too long for provisioned.
- Configure it per deployment, or per request with a header. It also works with Agent Service.
- Spilled requests are served, and paid for, as standard, pay-per-token traffic.

> ON THE EXAM
>
> Microsoft recommends enabling spillover on all global and data zone provisioned deployments.

Buy for the peak, or for the base?

PROVISIONED SIZED FOR PEAK

Always ready, often idle

- No throttling on busy days
- Paying for idle PTUs most of the time
- Hourly billing, used or not

PROVISIONED FOR BASE, SPILLOVER FOR PEAKS

Right-sized

- Steady load on reserved capacity
- Bursts overflow to pay-per-token
- Azure Reservations lower the base cost

THE TIEBREAKER

The capacity calculator on the Quota page estimates PTUs for your call shape. Reservations suit deployments kept for more than a few days.

Prompt caching

- **Automatic** for supported models. There is no setting to switch on.
- Needs a prompt of **at least 1,024 tokens**, with the **first 1,024 identical** to an earlier request.
- Put **stable content first**: instructions and examples. Put variable content last.
- Cached tokens are discounted on standard, and up to **100% off on provisioned**.
- One changed character near the start, such as a timestamp, means a cache miss.

> ON THE EXAM
>
> cached_tokens in the response usage shows whether caching is working. Zero means the prefix is not matching.

Where the money actually goes

LEVER	EFFECT
Smaller model for simple tasks	Often the largest saving (module 02)
Batch for work that can wait	50% cheaper than Global Standard (module 04)
Stable prompt prefix	Prompt caching discounts repeated input
Lower reasoning effort where acceptable	Fewer hidden reasoning tokens
Model router in Cost or Balanced mode	Cheaper models for easier prompts
Provisioned base with a reservation	Lower unit cost for steady volume

ON THE EXAM

Lowering max_tokens raises throughput, but only saves money if it actually shortens the answers generated.

admin@sthithapragna.com

The AI gateway in Azure API Management

- **llm-token-limit:** a token limit per consumer, such as per app or team. 429 over the rate, 403 over a quota.
- **llm-emit-token-metric:** token usage per consumer, for dashboards and chargeback.
- **Backend pools with priorities:** send traffic to provisioned first, then standard.
- **Circuit breaker:** stops sending to a throttled backend, honouring its Retry-After.
- **Semantic caching:** reuse responses for similar prompts.

> THE TIEBREAKER
>
> Deployment TPM limits the whole deployment. A gateway limits each app that shares it.

The cost footprint of an agent

- Agent Service does not charge for agent instances. You pay for what they use.
- One user question can mean **several model calls**: plan, call a tool, read the result, answer.
- **Conversation history** is resent each turn, so long conversations cost more per turn.
- Some tools, such as file search, carry **charges beyond model tokens**.
- Measure tokens per completed task, not per call, to see an agent's true cost.

> ON THE EXAM
>
> An agent that suddenly costs more may be looping through tools. Traces, in module 16, show every call.

> A legal firm's summarisation app receives frequent 429 errors. Azure Monitor shows billed token usage at about 20% of the deployment's TPM, and requests arrive evenly.
>
> Every request sets max_tokens to 16,000, although the summaries it returns are typically around 300 tokens long.

What should the team do first?

A Submit a quota increase request for the model in that region.

B Move the summarisation workload to a Global Batch deployment.

C Reduce max_tokens to a value close to the real summary length.

D Add retry logic with exponential backoff and leave the rest unchanged.

◆ PAUSE AND DECIDE

What should the team do first?

A Request more quota.

Treats an estimate problem as a quota problem. Usage is only at 20%.

B Move to Global Batch.

Summaries may be needed promptly, and batch does not fix the inflated estimate.

C Lower max_tokens to the real length.

The rate limit counts max_tokens, so 16,000 per call exhausts TPM long before billed usage does.

D Only add backoff.

Useful, but failed requests still count, and the root cause remains.

THE SIGNAL

Low billed usage plus 429s, with even traffic ruling out bursts, points straight at max_tokens.

> An airline's rebooking assistant runs on a Data Zone Provisioned deployment, sized for a normal day. It needs steady latency.
>
> On storm days, traffic triples for a few hours, the deployment returns 429s, and stranded passengers see failures. Finance will not fund capacity for storm peaks.

What should the airline do?

A. Triple the number of PTUs so the deployment can handle storm-day peaks.

B. Move the whole workload to a Data Zone Standard deployment instead.

C. Retry each 429 immediately in the client until the request succeeds.

D. Enable spillover to a standard deployment in the same Foundry resource.

◆ PAUSE AND DECIDE

What should the airline do?

A Triple the PTUs.

Solves the peaks, but pays hourly for idle capacity on every normal day. Finance said no.

B Move everything to standard.

Gives up the steady latency the assistant needs on normal days.

C Retry immediately until success.

Retries still count and add load, deepening the throttling at the worst moment.

D Enable spillover.

Normal load stays on provisioned; storm overflow is served as pay-per-token standard traffic.

THE SIGNAL

'Steady latency normally' plus 'no money for peaks' is the exact shape spillover was built for.

> Five internal apps share one model deployment. Marketing's app runs large jobs that use most of the deployment's TPM, and the HR helpdesk app starts getting 429s.
>
> The platform team needs a token limit for each app and a monthly token report per app, without creating separate deployments.

What should the platform team implement?

A. An API Management AI gateway with per-app token limits and metrics.

B. Five Foundry resources, giving each app its own deployment and quota.

C. A lower TPM allocation on the shared deployment, to slow all apps down.

D. A written policy asking the marketing team to run its jobs overnight.

◆ PAUSE AND DECIDE

What should the platform team implement?

A API Management AI gateway.

llm-token-limit caps each app; llm-emit-token-metric reports usage per app. One deployment.

B Five resources.

Isolates apps, but the requirement explicitly rules out separate deployments.

C Lower the shared TPM.

Punishes every app equally, and marketing can still crowd out HR.

D Ask marketing to run overnight.

Unenforced, and provides no per-app report.

THE SIGNAL

'Per app' limits on a shared deployment is a gateway requirement. Deployment settings cannot tell apps apart.

A legal assistant sends a 6,000-token block of firm policies and worked examples with every request.

Each prompt starts with the current timestamp and the user's question, followed by the policy block. The response always shows cached_tokens as zero.

What should the team change to benefit from prompt caching?

A Turn on prompt caching in the model deployment's advanced settings.

B Put the fixed policy block first, then the timestamp and question.

C Trim the policy block to under 1,024 tokens so it can be cached.

D Move the workload to a provisioned deployment to unlock caching.

◆ PAUSE AND DECIDE

What should the team change to benefit from prompt caching?

A Turn caching on.

There is no switch. Caching is automatic for supported models.

B Stable block first, variable content last.

The first 1,024 tokens become identical across requests, so they can be cached.

C Trim below 1,024 tokens.

Backwards. Prompts under 1,024 tokens cannot use caching at all.

D Move to provisioned.

Provisioned discounts cached tokens more, but a prefix that never matches never caches.

THE SIGNAL

A value that changes every request, placed at the start, guarantees a cache miss. Order the prompt from stable to variable.

Quotas, limits and cost in five lines

- **Quota, rate limit, cost:** three different numbers, counted at different times.
- **Rate limits count prompt plus max_tokens**, and requests in second-scale windows.
- **Provisioned base, spillover for peaks.** Reservations lower the base.
- **Retry with backoff, never in a tight loop.** Failed requests still count.
- **Stable prompts cache.** A gateway limits each app on a shared deployment.

> ON THE EXAM
>
> Next, module 07: securing AI systems with managed identity, keyless access, networking and roles.